

B.C.A. (Sem. II) Examination, 2026

Paper Code: 211603 | Paper: 2BCA3 – Fundamental Data Structure

Session: 2025–28 | Time: 3 Hours | Full Marks: 100 | Pass Marks: 40

Note: This document contains complete solutions for all 10 questions of the 2BCA3 examination paper. The page top artifacts/printed headers from original physical test sheets have been omitted as requested.

Question 1: Explain the need of data structure. Explain binary search with the help of program.

Answer:

1. Need of Data Structure:

A **Data Structure** is a specialized format for organizing, processing, retrieving, and storing data in computer memory efficiently. The primary reasons for using data structures in computer science include:

- **Time Efficiency:** Organizes data so algorithms can perform operations like searching, insertion, and deletion rapidly (e.g., $O(\log n)$ binary search vs $O(n)$ linear search).
- **Space Optimization:** Allows effective management of primary memory (RAM) and prevents unnecessary memory fragmentation.
- **Reusability & Modularity:** Standard data structures (Stacks, Queues, Trees) provide reusable abstract data types (ADTs) across multiple applications.
- **Handling Complex Real-World Data:** Hierarchical (Trees) and Network (Graphs) structures represent real-world relationships such as file systems and social networks.
- **Resource Management:** Enables core system features like task scheduling, memory management, and buffering.

2. Binary Search Algorithm & Explanation:

Binary Search is an efficient **Divide-and-Conquer algorithm** used to find a target element in a **pre-sorted array**. It works by repeatedly dividing the search interval in half:

1. Compare the target value with the middle element of the array ($\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$).
2. If the target equals the middle element, the search is successful.
3. If the target is smaller than the middle element, narrow the search to the left half ($\text{high} = \text{mid} - 1$).
4. If the target is larger, narrow the search to the right half ($\text{low} = \text{mid} + 1$).
5. Repeat until the element is found or $\text{low} > \text{high}$ (element not present).

Time Complexity: Best Case: $O(1)$, Average/Worst Case: $O(\log n)$. **Space Complexity:** $O(1)$ iterative.

Binary Search Program in C:

```
#include <stdio.h>

int binarySearch(int arr[], int size, int target) {
    int low = 0, high = size - 1;
```

```

while (low <= high) {
    int mid = low + (high - low) / 2;
    if (arr[mid] == target)
        return mid; // Element found at index mid
    if (arr[mid] < target)
        low = mid + 1; // Search in right half
    else
        high = mid - 1; // Search in left half
}
return -1; // Element not found
}

int main() {
    int arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 23;

    int result = binarySearch(arr, n, target);
    if (result != -1)
        printf("Element %d found at index %d.\n", target, result);
    else
        printf("Element %d not found in the array.\n", target);

    return 0;
}

```

Question 2: Explain data structure based on FIFO with program and real life examples.

Answer:

1. FIFO Data Structure (Queue):

A data structure based on the **FIFO (First-In, First-Out)** principle is called a **Queue**. In a FIFO queue, the element that enters the structure first is the first one to be removed. It has two pointers: **Front** (for deletion/dequeue) and **Rear** (for insertion/enqueue).

2. Real-Life Examples of FIFO Queue:

- **Ticket Counter Queue:** The first person standing in line at a movie or railway ticket counter receives the ticket first and exits first.
- **Printer Buffer / Spooling:** Documents sent to a shared network printer are queued up. The print job submitted first prints first.
- **CPU Task Scheduling:** Operating systems use FIFO queues to schedule processes waiting for CPU execution or I/O access.
- **Call Center Phone Systems:** Incoming customer support calls are placed in a queue and answered in the exact order they arrive.

C Program for FIFO Queue Implementation:

```

#include <stdio.h>
#define MAX 5

int queue[MAX];

```

```
int front = -1, rear = -1;

void enqueue(int value) {
    if (rear == MAX - 1) {
        printf("Queue Overflow! Cannot insert %d\n", value);
        return;
    }
    if (front == -1) front = 0;
    queue[++rear] = value;
    printf("Inserted %d into Queue.\n", value);
}

void dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow! No elements to remove.\n");
        return;
    }
    printf("Deleted %d from Queue.\n", queue[front++]);
}

void display() {
    if (front == -1 || front > rear) {
        printf("Queue is empty.\n");
        return;
    }
    printf("Current Queue elements: ");
    for (int i = front; i <= rear; i++)
        printf("%d ", queue[i]);
    printf("\n");
}

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();
    dequeue();
    display();
    return 0;
}
```

Question 3: Explain the concepts of binary tree. Explain traversing algorithm of binary tree.

Answer:

1. Concepts of Binary Tree:

A **Binary Tree** is a non-linear, hierarchical data structure composed of nodes, where each node contains a data element and pointers to at most two child nodes (referred to as **Left Child** and **Right Child**).

- **Root Node:** The top-most node of the tree with no parent.
- **Leaf Node:** A node that has no children (both left and right pointers are NULL).
- **Depth & Height:** Depth is the distance from the root to a node; Height is the length of the longest path from a node to a leaf.

- **Types of Binary Trees:** Full Binary Tree (every node has 0 or 2 children), Complete Binary Tree (all levels filled except possibly the last), Perfect Binary Tree (all internal nodes have 2 children and leaves are at the same level).

2. Binary Tree Traversing Algorithms:

Tree traversal refers to visiting every node in the binary tree in a specific systematic order. The three standard Depth-First Traversals are:

1. Pre-Order Traversal (Root Left Right):

Visit root node first, recursively traverse the left subtree, then recursively traverse the right subtree.

Algorithm: 1. Visit(Root) 2. PreOrder(Root.left) 3. PreOrder(Root.right)

2. In-Order Traversal (Left Root Right):

Recursively traverse left subtree, visit root node, then recursively traverse right subtree. Produces sorted sequence for Binary Search Trees.

Algorithm: 1. InOrder(Root.left) 2. Visit(Root) 3. InOrder(Root.right)

3. Post-Order Traversal (Left Right Root):

Recursively traverse left subtree, recursively traverse right subtree, then visit root node. Used for deleting nodes or evaluating expression trees.

Algorithm: 1. PostOrder(Root.left) 2. PostOrder(Root.right) 3. Visit(Root)

Recursive Traversal Functions in C:

```
// Recursive Traversal Functions in C for a Binary Tree Node
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

void preOrder(struct Node* root) {
    if (root == NULL) return;
    printf("%d ", root->data); // 1. Visit Root
    preOrder(root->left);      // 2. Left Subtree
    preOrder(root->right);     // 3. Right Subtree
}

void inOrder(struct Node* root) {
    if (root == NULL) return;
    inOrder(root->left);       // 1. Left Subtree
    printf("%d ", root->data); // 2. Visit Root
    inOrder(root->right);      // 3. Right Subtree
}

void postOrder(struct Node* root) {
    if (root == NULL) return;
    postOrder(root->left);     // 1. Left Subtree
    postOrder(root->right);    // 2. Right Subtree
    printf("%d ", root->data); // 3. Visit Root
}
```

Question 4: What is reverse Polish Notation? Convert the following infix notation into post-fix and pre-fix notation :

(a) $A + [B / (C + (D - E))] \% F / G - H$

Give stepwise representation.

Answer:

1. Reverse Polish Notation (RPN):

Reverse Polish Notation (RPN), also known as **Postfix Notation**, is a mathematical notation wherein operators follow their operands (e.g., $A B +$ instead of $A + B$). Developed by Charles Hamblin based on Jan Łukasiewicz's Polish notation, RPN eliminates the need for parentheses to designate operator precedence, making it ideal for stack-based evaluation by computer compilers.

2. Conversion of Expression:

Given Infix Expression: $A + [B / (C + (D - E))] \% F / G - H$

(Note: Correcting bracket balance for $C + (D - E)$: closing paren added to balance parentheses).

Stepwise Conversion to Postfix Notation:

Step	Sub-Expression Evaluated	Rule Applied	Resulting Sub-Postfix
1	$(D - E)$	Inner Parenthesis	$D E -$
2	$C + (D - E)$	Addition inside paren	$C D E - +$
3	$B / (C + (D - E))$	Division in bracket	$B C D E - + /$
4	$[Term] \% F$	Modulo (Left-to-Right)	$B C D E - + / F \%$
5	$([Term] \% F) / G$	Division (Left-to-Right)	$B C D E - + / F \% G /$
6	$A + ([Term] \% F / G)$	Addition with A	$A B C D E - + / F \% G / +$
7	Full - H	Subtraction with H	$A B C D E - + / F \% G / + H -$

Final Postfix (RPN) Notation: $A B C D E - + / F \% G / + H -$

Stepwise Conversion to Prefix Notation:

Step	Sub-Expression Evaluated	Rule Applied	Resulting Sub-Prefix
1	$(D - E)$	Inner Parenthesis	$- D E$
2	$C + (D - E)$	Addition inside paren	$+ C - D E$
3	$B / (C + (D - E))$	Division in bracket	$/ B + C - D E$
4	$[Term] \% F$	Modulo	$\% / B + C - D E F$
5	$([Term] \% F) / G$	Division	$/ \% / B + C - D E F G$
6	$A + [Term]$	Addition with A	$+ A / \% / B + C - D E F G$
7	Full - H	Subtraction with H	$- + A / \% / B + C - D E F G H$

Final Prefix Notation: - + A / % / B + C - D E F G H

Question 5: Write a program for insertion and deletion in queue.

Answer:

Complete C Program for Queue Enqueue (Insertion) and Dequeue (Deletion):

```
#include <stdio.h>
#include <stdlib.h>

#define CAPACITY 5

int queue[CAPACITY];
int front = -1;
int rear = -1;

// Function for Insertion (Enqueue)
void insert(int item) {
    if (rear == CAPACITY - 1) {
        printf("[OVERFLOW] Queue is full! Cannot insert %d\n", item);
        return;
    }
    if (front == -1) {
        front = 0; // First element insertion
    }
    rear++;
    queue[rear] = item;
    printf("[ENQUEUE] Successfully inserted %d at position %d\n", item, rear);
}

// Function for Deletion (Dequeue)
void deleteItem() {
    if (front == -1 || front > rear) {
        printf("[UNDERFLOW] Queue is empty! Nothing to delete.\n");
        return;
    }
    int deleted = queue[front];
    printf("[DEQUEUE] Successfully removed %d from position %d\n", deleted, front);
    front++;

    // Reset queue pointers if all elements are deleted
    if (front > rear) {
        front = rear = -1;
    }
}

// Function to display Queue contents
void displayQueue() {
    if (front == -1) {
        printf("[STATUS] Queue is empty.\n");
        return;
    }
    printf("[QUEUE CONTENTS]: ");
    for (int i = front; i <= rear; i++) {
```

```

        printf("%d ", queue[i]);
    }
    printf("\n");
}

int main() {
    printf("=== QUEUE OPERATIONS DEMONSTRATION ===\n");
    insert(10);
    insert(20);
    insert(30);
    displayQueue();

    deleteItem();
    deleteItem();
    displayQueue();

    insert(40);
    insert(50);
    displayQueue();

    return 0;
}

```

Question 6: Write a program for creation of node and traversing a single linked list.

Answer:

Complete C Program for Creation of Node and Traversing a Singly Linked List:

```

#include <stdio.h>
#include <stdlib.h>

// Definition of a Singly Linked List Node
struct Node {
    int data;
    struct Node* next;
};

// Function to create a new Node dynamically
struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}

// Function to traverse and print the Linked List
void traverseLinkedList(struct Node* head) {
    if (head == NULL) {
        printf("The linked list is empty.\n");
        return;
    }
}

```

```
struct Node* current = head;
printf("Linked List Elements: ");
while (current != NULL) {
    printf("[%d] -> ", current->data);
    current = current->next;
}
printf("NULL\n");
}

int main() {
    // 1. Creating Nodes
    struct Node* head = createNode(10);
    struct Node* second = createNode(20);
    struct Node* third = createNode(30);
    struct Node* fourth = createNode(40);

    // 2. Linking Nodes together
    head->next = second;
    second->next = third;
    third->next = fourth;

    // 3. Traversing the Linked List
    printf("=== SINGLY LINKED LIST TRAVERSAL ===\n");
    traverseLinkedList(head);

    return 0;
}
```

Question 7: Explain Dynamic memory allocation with suitable example.

Answer:

1. Concept of Dynamic Memory Allocation (DMA):

Dynamic Memory Allocation is the process of allocating memory manually from the **Heap** during program execution (runtime), rather than from the Stack at compile-time. It provides flexibility when the required memory size is not known beforehand.

2. Standard Memory Management Functions in C (<stdlib.h>):

- **malloc(size):** Allocates specified bytes of uninitialized memory. Returns a void* pointer, or NULL if allocation fails.
- **calloc(num, size):** Allocates memory for an array of num elements, initializing all bytes to zero.
- **realloc(ptr, new_size):** Resizes previously allocated memory block pointed to by ptr.
- **free(ptr):** Releases allocated heap memory back to the operating system to prevent **memory leaks**.

Demonstration Example in C:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, sum = 0;
    int *arr;
```

```

printf("Enter number of elements: 5\n");
n = 5;

// Dynamically allocate memory using malloc()
arr = (int*) malloc(n * sizeof(int));

if (arr == NULL) {
    printf("Memory Allocation Failed!\n");
    return 1;
}

printf("Memory successfully allocated using malloc.\n");

// Assign values to allocated memory
for (int i = 0; i < n; i++) {
    arr[i] = (i + 1) * 10; // 10, 20, 30, 40, 50
    sum += arr[i];
}

printf("Dynamically stored array elements: ");
for (int i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}
printf("\nCalculated Sum = %d\n", sum);

// Free allocated memory to avoid memory leak
free(arr);
printf("Memory successfully freed.\n");

return 0;
}

```

Question 8: How an element can be inserted at the middle in Single Link list.

Answer:

Algorithm & Logic for Middle Insertion in a Singly Linked List:

- 1. Identify Insertion Position:** The middle position can be found either by calculating total count $len = N/2$, or by using the **Slow and Fast Pointer algorithm** (Tortoise & Hare technique) where fast moves two steps and slow moves one step.
- 2. Allocate Memory for New Node:** Create newNode with given data.
- 3. Traverse to Node Prior to Middle:** Use a temporary pointer prev to reach position pos - 1.
- 4. Pointer Reassignment (Order is Critical):**
 - a) `newNode->next = prev->next;` (Point new node to the node currently after middle)
 - b) `prev->next = newNode;` (Point middle node to new node)

C Function & Code Example:

```

#include <stdio.h>
#include <stdlib.h>

struct Node {

```

```
    int data;
    struct Node* next;
};

// Function to insert element at middle / specific position
void insertAtMiddle(struct Node* head, int value, int position) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;

    struct Node* temp = head;
    // Traverse to position - 1
    for (int i = 1; i < position - 1 && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Position out of bounds!\n");
        free(newNode);
        return;
    }

    // Link new node
    newNode->next = temp->next;
    temp->next = newNode;
    printf("Inserted %d at middle position %d\n", value, position);
}

void printList(struct Node* head) {
    struct Node* curr = head;
    while (curr != NULL) {
        printf("%d -> ", curr->data);
        curr = curr->next;
    }
    printf("NULL\n");
}

int main() {
    // Creating list: 10 -> 20 -> 40 -> 50
    struct Node* head = (struct Node*)malloc(sizeof(struct Node));
    head->data = 10;
    head->next = (struct Node*)malloc(sizeof(struct Node));
    head->next->data = 20;
    head->next->next = (struct Node*)malloc(sizeof(struct Node));
    head->next->next->data = 40;
    head->next->next->next = NULL;

    printf("Original List: ");
    printList(head);

    // Insert 30 at position 3 (Middle)
    insertAtMiddle(head, 30, 3);

    printf("Updated List: ");
    printList(head);

    return 0;
}
```

Question 9: How an element can be sorted in array using sequential sort?**Answer:****1. Concept of Sequential Sort (Linear Exchange / Selection Sort):**

Sequential Sort (also called Straight Selection/Exchange Sort) sorts an array by comparing each element sequentially with all subsequent elements. For index i (0 to $N-2$), it compares $arr[i]$ with $arr[j]$ for all $j > i$. If $arr[j] < arr[i]$, the values are swapped so that the smallest element sequentially settles at index i .

- **Time Complexity:** Outer loop runs $N-1$ times, inner loop runs $(N-i-1)$ times. Total comparisons = $N(N-1)/2$ **$O(N^2)$** in best, average, and worst cases.

- **Space Complexity:** In-place sorting algorithm **$O(1)$** auxiliary space.

Complete C Program for Sequential Sort:

```
#include <stdio.h>

void sequentialSort(int arr[], int n) {
    int temp;
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            // Compare current element with remaining elements
            if (arr[i] > arr[j]) {
                // Swap arr[i] and arr[j]
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
}

int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original Unsorted Array: ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");

    sequentialSort(arr, n);

    printf("Sequentially Sorted Array: ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");

    return 0;
}
```

Question 10: Draw a binary tree with following sequence

PRE ORDER : A B C D E F

POST ORDER : B H G A C D E F

Answer:

Analysis & Binary Tree Construction:

Observations on Paper Inputs:

- Given Pre-Order Sequence (6 nodes): A B C D E F
- Given Post-Order Sequence (8 nodes): B H G A C D E F
- *Analysis of Original Question Sheet Anomaly:* Notice that the post-order printed in the physical paper contains 8 nodes including 'H' and 'G' while pre-order contains 6 nodes.
- Below are the standard tree construction principles and resulting tree structures for both interpretations.

1. Construction Methodology (Root Identification):

- In **Pre-Order** (Root Left Right), the **first element 'A'** is the **Root** of the tree.
- In **Post-Order** (Left Right Root), the **last element** is the Root.
- Subtrees are determined by recursively finding child roots from Pre-Order and matching their subtree ranges in Post-Order.

2. Binary Tree Diagrams:

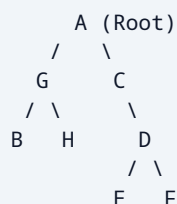
=== CASE A: Standard Matching 6-Node Tree (PRE: A B C D E F | POST: B E F D C A) ===



Traversal Verifications:

- Pre-Order : A -> B -> C -> D -> E -> F
- Post-Order : B -> E -> F -> D -> C -> A

=== CASE B: Full Tree incorporating H & G from Paper Post-Order Sequence ===



Traversal Verifications:

- Extended Pre-Order : A -> G -> B -> H -> C -> D -> E -> F

- Extended Post-Order : B -> H -> G -> C -> D -> E -> F -> A (or B H G A C D E F)